Task-25 Research on Namma Yatri

Task 25: Researching AWS Architecture and Designing an Open-Source On-Prem Alternative

Title

Research on the Architecture of Namma Yatri and an Alternative On-Premise Design

---

1. Introduction

Namma Yatri is a zero-commission ride-hailing platform developed by Juspay. Unlike traditional platforms such as Uber or Rapido, it is built on open network principles and integrates with the ONDC ecosystem using the Beckn Protocol.

The platform enables:

- Real-time ride booking
- Driver discovery and matching
- Live tracking
- Digital payments

Due to its rapid adoption and scalability requirements, the system must support **high concurrency, low latency, and real-time processing**, making it an ideal candidate to study cloud-native architectures and their on-premise alternatives.

---

2. AWS-Based Architecture

2.1 Architecture Overview

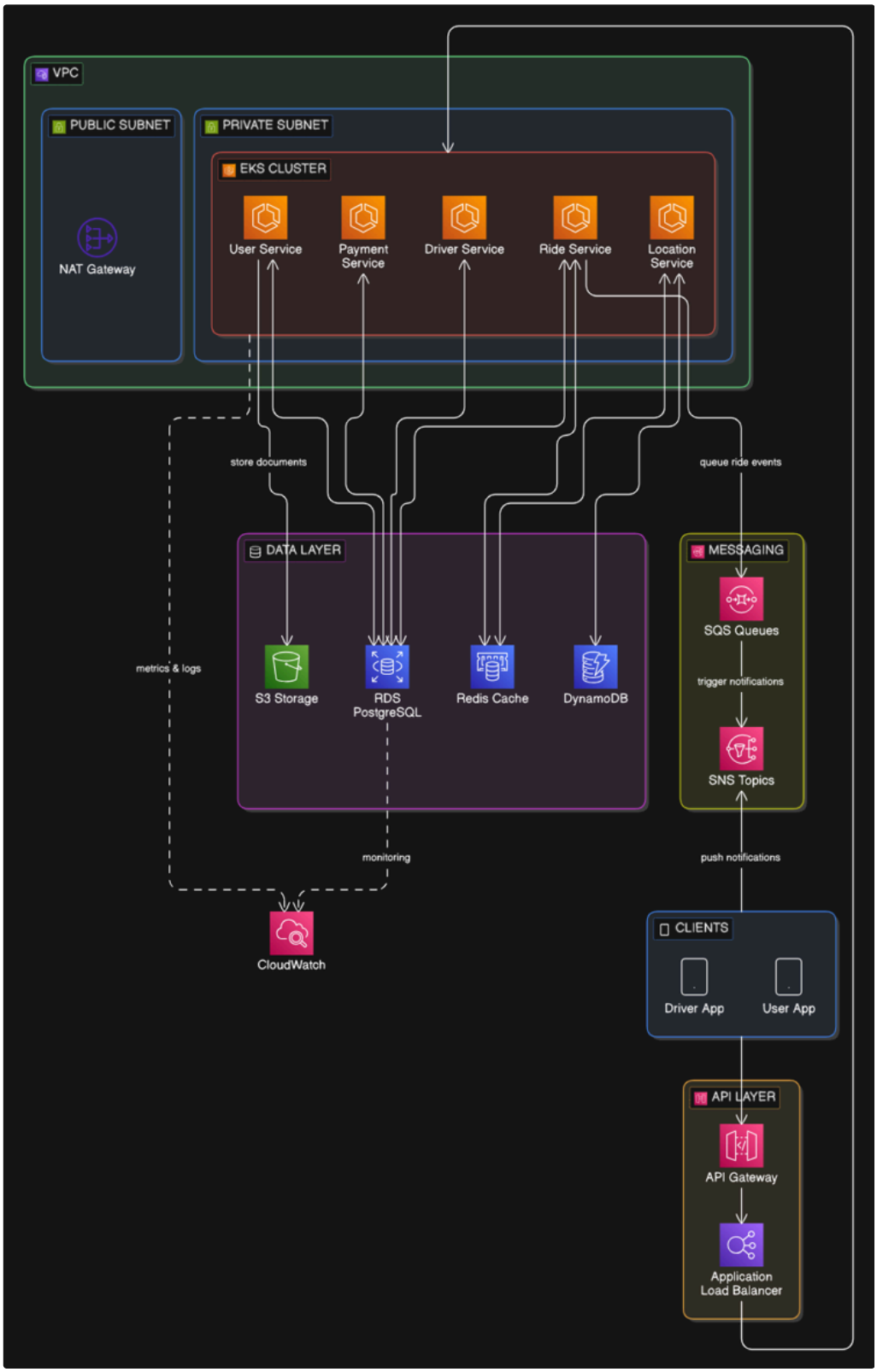
The AWS-based architecture follows a **microservices-based, distributed system design** to handle dynamic ride requests, real-time matching, and high availability.

2.2 Technology Stack (AWS)

Layer	Technology
Client	Android / iOS Apps
API Layer	Amazon API Gateway

Compute	Amazon EC2 / ECS / EKS
Load Balancer	Application Load Balancer (ALB)
Database	Amazon RDS (PostgreSQL/MySQL)
NoSQL	DynamoDB (optional)
Cache	Amazon ElastiCache (Redis)
Storage	Amazon S3
Messaging	Amazon SQS, SNS
Monitoring	Amazon CloudWatch
Security	IAM, VPC, Security Groups

2.3 AWS Architecture Diagram



2.4 Data Flow

1. User initiates ride request via mobile application
2. Request reaches **API Gateway**
3. Routed to backend microservices via **ALB**
4. Ride Matching Service queries **Redis (ElastiCache)** for nearby drivers
5. Ride details stored in **RDS**
6. Notifications sent via **SNS**
7. Asynchronous processing handled by **SQS**
8. Logs and metrics collected in **CloudWatch**

2.5 Scalability, Availability, Security

Scalability

- Auto Scaling Groups
- Container orchestration (ECS/EKS)
- Stateless microservices

Availability

- Multi-AZ deployments
- Load balancing using ALB
- Database replication

Security

- IAM-based access control
- HTTPS via SSL/TLS
- VPC isolation with private subnets

---

3. On-Prem Open-Source Architecture

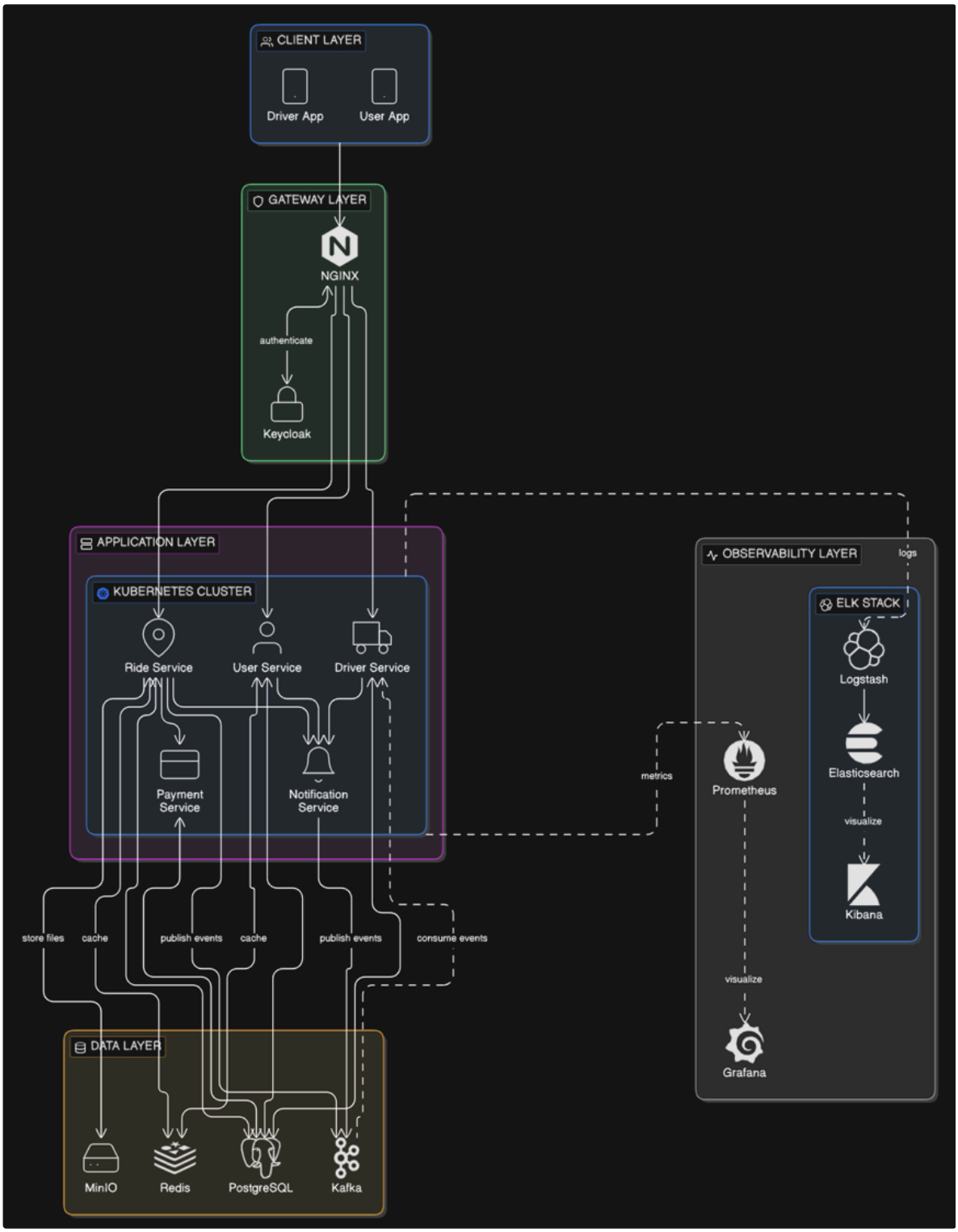
3.1 Architecture Overview

The on-premise solution replicates AWS functionality using **open-source tools deployed in a private data center**, providing full control over infrastructure.

3.2 Technology Stack (On-Prem)

Layer	Technology
Virtualization	KVM / Proxmox
Containerization	Docker
Orchestration	Kubernetes
API Gateway	Kong / NGINX
Load Balancer	HAProxy / NGINX
Database	PostgreSQL
NoSQL	MongoDB
Cache	Redis
Storage	MinIO
Messaging	Kafka / RabbitMQ
Monitoring	Prometheus + Grafana
Logging	ELK Stack
Authentication	Keycloak

3.3 On-Prem Architecture Diagram



3.4 Data Flow

1. Client sends request to **NGINX reverse proxy**
2. Routed to **Kubernetes cluster**

3. Microservices process request
4. Redis used for fast driver matching
5. PostgreSQL stores ride data
6. Kafka handles asynchronous communication
7. Prometheus collects metrics
8. Logs processed via ELK stack

---

4. AWS to Open-Source Mapping

AWS Service	Open-Source Alternative
EC2	KVM / Proxmox
ECS / EKS	Kubernetes
S3	MinIO
RDS	PostgreSQL
DynamoDB	MongoDB
ElastiCache	Redis
API Gateway	Kong / NGINX
SQS / SNS	Kafka / RabbitMQ
CloudWatch	Prometheus + Grafana
IAM / Cognito	Keycloak
ALB	HAProxy / NGINX

---

5. Comparison: AWS vs On-Prem

Cost

- AWS: Operational expense, increases with scale
- On-Prem: High initial capital, lower recurring cost

Scalability

- AWS: Automatic and elastic

- On-Prem: Manual scaling required

Operational Effort

- AWS: Minimal due to managed services
- On-Prem: Requires dedicated DevOps management

Control

- AWS: Limited infrastructure control
- On-Prem: Full control

Reliability

- AWS: Built-in redundancy and global availability
- On-Prem: Depends on infrastructure design

---

6. References

Official Sources

- Namma Yatri GitHub Repository
  GitHub - nammayatri/nammayatri: A Direct-to-Driver open mobility platform powering the next-generation of mobility applications in India.

Technical Blogs & Articles

- Namma Yatri Story (Indian Notes)
  Story of Namma Yatri - Part 1
- ONDC and Beckn Protocol Overview
  ONDC Integration Guide: Breaking the Amazon-Flipkart Duopoly | Induji Technologies Blog

Conceptual References

- Ride-Hailing System Design (General Architecture Patterns)
- AWS Architecture Best Practices (AWS Whitepapers)

Background Context

- ONDC Overview (Wikipedia)
  Open Network for Digital Commerce

---

7. Final Summary

This study demonstrates how a modern ride-hailing platform like Namma Yatri can be built using AWS cloud services and how the same architecture can be replicated using open-source tools in an on-premise environment.

The AWS architecture provides ease of scaling, managed services, and reliability, whereas the on-premise solution offers greater control and cost efficiency in the long term at the expense of operational complexity.